

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет информатики, математики и компьютерных наук
Образовательная программа «Компьютерные науки и технологии»

Ким Игорь Геннадьевич, группа 23КНТ-4

**РАЗРАБОТКА СИСТЕМЫ БЕСПРОВОДНОГО УПРАВЛЕНИЯ
ГИРОСТАБИЛИЗИРУЕМОЙ ПЛАТФОРМОЙ**

Курсовая работа

Научный руководитель:

Морозов Никита Сергеевич, научный сотрудник, НГТУ им. Р.Е. Алексеева
(НГТУ)

Нижний Новгород, 2026

Оглавление

Введение	5
Цель и задачи работы	6
Роль автора и состав выполненных работ	8
Глава 1. Исследовательская часть	9
1.1. Актуальность и предметная область	9
1.2. Назначение гиросtabilизируемых платформ	10
1.3. Анализ требований к прототипу	11
1.4. Анализ технических подходов и аналогов	12
1.5. Обоснование выбранной аппаратно-программной архитектуры	14
Глава 2. Проектирование системы	15
2.1. Общая архитектура системы	15
2.2. Аппаратная часть	16
2.3. Подсистема инерциальных измерений	17
2.4. Подсистема исполнительных механизмов	18
2.5. Беспроводной интерфейс и удаленное управление	19
2.6. Механическая часть и 3D-печатный корпус	20
Глава 3. Реализация программно-аппаратного прототипа	21
3.1. Embedded-часть на ESP32-P4-M3	21

3.2. Работа с MPU-9250 по I2C	22
3.3. Телеметрия, HTTP API и WebSocket	23
3.4. Backend и frontend станции управления	24
3.5. Обработка команд управления приводом	25
Глава 4. Алгоритмы обработки данных и стабилизации	27
4.1. Модель обработки измерений IMU	27
4.2. Оценка угла наклона и фильтрация	28
4.3. Регулятор компенсации наклона	29
4.4. Ограничения текущей реализации	30
Глава 5. Испытания и анализ результатов	31
5.1. Методика испытаний	31
5.2. Проверка аппаратных подключений	32
5.3. Проверка беспроводного интерфейса	33
5.4. Проверка пользовательского интерфейса	34

Листинг 1 — Фрагмент UART-лога при старте прошивки (реальный вывод):

```

I (401) H_SDIO_DRV: sdio_data_to_rx_buf_task started
I (421) main_task: Calling app_main()
I (421) app: -----
I (421) app:   APP INITIALIZATION
I (421) app: -----
I (441) i2c_bus: scan: found 1 device(s)
I (441) app: mpu: addr=0x68 WHO_AM_I=0x71 (MPU-9250)
I (5231) app_wifi: AP ready ssid='JC-ESP32P4M3' ip='192.168.4.1'
I (5241) app_net: http/ws server listening on port 80

```

```
I (5251) app_ble: advertising started as 'JC-P4-BLE'
@telemetry {"kind":"system","uptime_ms":5300,"firmware":"hello_world_p4"}
@telemetry {"kind":"control","active":false,"angle_deg":178.66,"target_deg":178.66}
# После команды 'g' (режим стабилизации):
@telemetry
{"kind":"control","active":true,"angle_deg":178.65,"error_deg":0.01,"p":0.020,"i":0.001,"d":0.000,"output":0.021,"dt_ms":20.0}
```

5.5. Испытания стабилизации 34

5.6. Испытания стабилизации 35

Глава 6. Экономическое обоснование и охрана труда 36

Заключение 39

Список использованных источников 40

Приложения 41

После внесения финальных рисунков и таблиц оглавление должно быть обновлено автоматическими средствами Word непосредственно перед экспортом итоговой версии документа.

Введение

Системы стабилизации положения относятся к классу программно-аппаратных систем, в которых результат определяется не только качеством отдельного алгоритма, но и согласованной работой механики, датчиков, исполнительных механизмов, микроконтроллера и пользовательского интерфейса. Гиростабилизированные платформы применяются в робототехнике, мобильных измерительных комплексах, системах ориентации камер, малых беспилотных устройствах и учебных лабораторных стендах. Их задача состоит в том, чтобы измерять текущее положение объекта, оценивать отклонение от требуемого состояния и формировать управляющее воздействие, компенсирующее наклон или внешнее возмущение.

Актуальность выбранной темы связана с тем, что современные встраиваемые системы все чаще строятся как распределенные комплексы: вычислительное ядро работает с датчиками и приводами, а управление, диагностика и визуализация выполняются через беспроводной интерфейс. Такой подход повышает удобство отладки и эксплуатации, но требует продуманной архитектуры обмена данными, устойчивого протокола управления, достаточной частоты телеметрии и понятного пользовательского интерфейса. В учебном проекте это особенно важно, потому что система должна быть не только работоспособной, но и объяснимой: каждый аппаратный и программный узел должен быть проверяем измерением, логом или наблюдаемым поведением.

В рамках данной курсовой работы рассматривается разработка прототипа системы беспроводного управления гиростабилизированной платформой на базе микроконтроллерной платы JC-ESP32P4-M3 и инерциального датчика GY-9250 / MPU-9250. В проекте реализуются слои подключения IMU по I2C, первичной телеметрии, беспроводного доступа, интерфейса удаленного

управления, а также механическая часть в виде напечатанного на 3D-принтере сферического элемента, подготовленного в Blender. Отдельное внимание уделяется разделению фактически реализованной части и тех фрагментов, которые требуют завершения перед финальной защитой.

Работа относится к проектному типу курсовой работы, поскольку ее результатом является не только теоретический анализ, но и программно-аппаратный прототип. При этом исследовательская часть также является существенной: необходимо проанализировать предметную область, выбрать аппаратную платформу, описать датчики ориентации, способы фильтрации измерений, принципы замкнутого управления и методику проверки результата. Такой формат позволяет показать не отдельный фрагмент кода, а полный инженерный путь от постановки задачи до тестируемого стенда.

На момент подготовки данного текста подтверждено подключение инерциального датчика к плате: линия SDA соответствует GPIO1, линия SCL соответствует GPIO2, адрес устройства на шине I2C равен 0x68, а регистр WHO_AM_I возвращает значение 0x71, соответствующее MPU-9250. Также подготовлена структура удаленного управления и телеметрии, включающая embedded-часть, backend-модуль и frontend-интерфейс оператора. Для разделов, где пока отсутствуют численные результаты финальных испытаний, в тексте даны явные словесные указания о том, какие измерения и материалы необходимо добавить перед защитой.

Цель и задачи работы

Целью курсовой работы является разработка и документирование прототипа системы беспроводного управления гиросtabilизируемой платформой, включающего аппаратный стенд, подсистему инерциальных

измерений, программные модули телеметрии и управления, а также механический корпусной элемент, изготовленный с применением 3D-печати.

Для достижения цели были сформулированы следующие задачи:

- проанализировать предметную область гиростабилизируемых платформ и выделить требования к учебному прототипу;
- выбрать состав аппаратных компонентов и определить их роль в системе;
- реализовать подключение инерциального датчика GY-9250 / MPU-9250 к JC-ESP32P4-M3 по интерфейсу I2C;
- реализовать получение и публикацию телеметрии датчика в формате, пригодном для дальнейшей обработки и визуализации;
- разработать беспроводной контур удаленного управления и диагностики на базе HTTP API и WebSocket;
- подготовить пользовательский интерфейс для отображения состояния системы и передачи команд исполнительному механизму;
- разработать и изготовить механическую часть прототипа в виде сферического элемента, смоделированного в Blender и распечатанного на 3D-принтере;
- описать алгоритмическую основу стабилизации и выделить этапы, требующие финальной интеграции;
- сформировать программу испытаний и обозначить недостающие экспериментальные данные, которые необходимо получить перед защитой.

Объектом работы является программно-аппаратная система управления положением платформы. Предметом работы являются методы беспроводного управления, обработки данных инерциального датчика и организации программной архитектуры прототипа гиростабилизации. Практическая

значимость работы состоит в создании стенда, который может быть использован для дальнейших экспериментов с алгоритмами стабилизации, анализа телеметрии и отладки встраиваемого программного обеспечения.

Роль автора и состав выполненных работ

В рамках проекта автор выполнял задачи, связанные с интеграцией аппаратных модулей, программной архитектурой embedded-части, организацией удаленного управления и подготовкой физического прототипа. Основной фокус работы был направлен на то, чтобы связать отдельные компоненты системы в проверяемую цепочку: датчик ориентации - микроконтроллер - телеметрия - интерфейс оператора - команды управления - физический стенд.

К выполненным работам относятся: анализ подключений GY-9250 / MPU-9250, настройка линий I2C, проверка адреса устройства, чтение идентификатора WHO_AM_I, подготовка логов диагностики, разработка модулей телеметрии, настройка сетевого взаимодействия, разработка интерфейса управления и изготовление сферического элемента корпуса. Отдельно была проведена работа по проектированию механической части в Blender и последующей 3D-печати, что позволило перейти от чисто программного прототипа к физически собираемому стенду.

Границы ответственности в данной версии документа определены следующим образом: в тексте уверенно описываются только те части, которые подтверждены кодом, логами, схемой подключения или фактом изготовления. Для финального замкнутого контура стабилизации и количественных испытаний оставлены специальные пометки, так как эти данные должны быть получены после финального стендового прогона. Такой подход выбран для того, чтобы работа оставалась академически корректной и не содержала неподтвержденных заявлений.

Глава 1. Исследовательская часть

1.1. Актуальность и предметная область

Гиростабилизируемая платформа является примером мехатронной системы, в которой одновременно решаются задачи измерения, вычисления, управления и физического воздействия на объект. В отличие от простых систем дистанционного управления, где команда оператора напрямую передается исполнительному механизму, в стабилизируемой системе требуется учитывать текущее состояние платформы. Система должна понимать, как она ориентирована в пространстве, какой угол отклонения присутствует и какое воздействие необходимо сформировать для компенсации.

Ключевая особенность таких платформ состоит в замкнутом контуре управления. Датчик ориентации измеряет ускорения и угловые скорости, программный модуль оценивает положение, регулятор вычисляет управляющее воздействие, а привод изменяет состояние механической части. После этого датчик снова фиксирует новое положение. Если этот цикл работает достаточно быстро и устойчиво, платформа способна сопротивляться внешним возмущениям. Если же один из слоев работает неправильно, система становится нестабильной или непредсказуемой.

Для учебного проекта гиростабилизируемая платформа удобна тем, что в ней наглядно проявляются все уровни embedded-разработки. На электрическом уровне нужно обеспечить питание, общую землю, корректные логические уровни и надежную шину I2C. На уровне периферии микроконтроллера нужно настроить GPIO, I2C, сетевой стек и тайминги. На уровне прошивки требуется организовать чтение датчика, обработку ошибок и публикацию телеметрии. На уровне прикладного программного обеспечения необходимо отобразить данные

пользователю и передать команды управления. На механическом уровне нужно обеспечить возможность установки платы, датчика, привода и корпуса.

Тем самым данная курсовая работа находится на стыке робототехники, микроконтроллерного программирования, сетевых протоколов, интерфейсной разработки и быстрого прототипирования. Это делает проект показательным с точки зрения инженерной подготовки: результат нельзя получить только написанием программы или только сборкой схемы. Необходима проверяемая интеграция всех слоев системы.

1.2. Назначение гиростабилизируемых платформ

Гиростабилизируемые платформы применяются там, где требуется сохранять ориентацию полезной нагрузки при движении основания. В малых робототехнических системах это может быть стабилизация камеры, датчика расстояния, антенны или измерительного модуля. В более сложных системах используются многокоординатные подвесы, которые компенсируют повороты по нескольким осям. В учебном прототипе основное внимание уделяется не промышленной точности, а построению принципиально правильной архитектуры управления.

С функциональной точки зрения стабилизируемая платформа должна выполнять четыре действия. Во-первых, она должна измерять свое состояние. Во-вторых, она должна преобразовывать сырые измерения в физически интерпретируемые параметры: угол, скорость, направление отклонения. В-третьих, она должна вычислять корректирующее воздействие. В-четвертых, она должна воздействовать на исполнительный механизм и контролировать результат. Именно эта цепочка определяет содержание проектной части работы.

Важной особенностью является то, что стабилизация зависит от частоты обновления данных. Если измерения поступают редко, регулятор получает устаревшую информацию. Если команды передаются с большой задержкой,

механическая часть не успевает компенсировать наклон. Если телеметрия нестабильна, оператор не может оценить состояние системы. Поэтому в работе рассматриваются не только сами алгоритмы, но и инфраструктура передачи данных: HTTP API, WebSocket, backend-мост и frontend-интерфейс.

1.3. Анализ требований к прототипу

Требования к прототипу можно разделить на функциональные, аппаратные, программные, эксплуатационные и документируемые. Такое разделение важно, потому что курсовая работа оценивает не только наличие работающего устройства, но и полноту обоснования инженерных решений.

Группа требований	Содержание
Функциональные требования	Считывание данных IMU, передача телеметрии, прием команд управления, отображение состояния системы, подготовка контура стабилизации.
Аппаратные требования	Совместимость с JC-ESP32P4-M3, подключение GY-9250 / MPU-9250 по I2C, корректное питание 3.3 В, общий GND, физическая установка внутри корпуса.
Программные требования	Модульная embedded-архитектура, обработка ошибок I2C, сетевой API, журналирование, возможность дальнейшего расширения алгоритма стабилизации.
Эксплуатационные требования	Удобный запуск, возможность наблюдения логов, доступ к интерфейсу управления, наличие методики испытаний.
Документируемые требования	Описание архитектуры, схемы подключений, алгоритмов, испытаний, ограничений и недостающих результатов.

Для текущего этапа проекта наиболее критичными являются требования к наблюдаемости системы. Прототип должен быть устроен так, чтобы любую ошибку можно было локализовать: отдельно проверить питание, отдельно шину I2C, отдельно наличие устройства на адресе 0x68, отдельно чтение WHO_AM_I,

отдельно получение телеметрии, отдельно сетевой доступ и отдельно реакцию интерфейса на команды пользователя. Такой подход снижает риск ситуации, когда система “не работает”, но неясно, где именно находится ошибка.

С точки зрения защиты курсовой работы требования к результату нужно формулировать осторожно. Уже реализованные части можно описывать как подтвержденный результат. Части, для которых нет численных испытаний, нужно описывать как подготовленные к интеграции или как требующие финального стендового прогона. Это особенно относится к разделу стабилизации, где важно иметь не только код, но и измеримые показатели: угол отклонения, время установления, перерегулирование, устойчивость телеметрии и задержку управляющей команды.

1.4. Анализ технических подходов и аналогов

Возможны разные подходы к построению гиросtabilизируемой платформы. Наиболее простой вариант - использовать готовый контроллер стабилизации или готовый подвес. Такой подход сокращает время разработки, но плохо подходит для учебной работы: большая часть алгоритмов и аппаратных решений скрыта внутри готового изделия. Второй вариант - построить собственную систему на микроконтроллере, инерциальном датчике и приводе. Он требует больше работы, но позволяет показать архитектуру, код, схему подключения и методику испытаний.

В рамках данной работы выбран второй подход. Он дает возможность контролировать все ключевые уровни системы: датчик, шину данных, прошивку, сетевой интерфейс, пользовательский интерфейс и механическую часть. Это соответствует проектному характеру курсовой работы: результатом является не покупка готового решения, а разработка собственного прототипа с документированными инженерными решениями.

Для оценки ориентации в малых системах часто используются инерциальные датчики семейства MPU, включающие акселерометр и гироскоп. Акселерометр позволяет оценивать направление вектора тяжести при медленных изменениях положения, но чувствителен к вибрациям и ускорениям движения. Гироскоп измеряет угловую скорость и хорошо описывает быстрые изменения, но при интегрировании накапливает дрейф. Поэтому в реальных системах данные обычно объединяются фильтром, например комплементарным фильтром, фильтром Мэдживика, фильтром Махони или Калмановским подходом.

Для беспроводного управления также возможны разные варианты: Bluetooth, Wi-Fi в режиме подключения к роутеру, Wi-Fi в режиме точки доступа, WebSocket, UDP или собственный бинарный протокол. В текущей реализации для JC-ESP32P4-M3 выбран hosted Wi-Fi контур с ESP32-C6 и HTTP/WebSocket API, потому что он лучше всего сочетается с web-интерфейсом, позволяет быстро отлаживать команды через браузер и упрощает демонстрацию результата на защите. Для данной hosted-конфигурации Bluetooth на сопроцессоре штатно не поддерживается, поэтому Bluetooth рассматривается как альтернативный вариант для отдельной версии прошивки, а не как подтвержденный транспорт текущей сборки. При этом для жестких real-time задач HTTP/WebSocket-подход не является идеальным, но для прототипа и диагностического контура он оправдан.

Подход	Преимущества	Ограничения	Вывод
Готовый подвес	Быстрый результат, промышленная механика	Скрытая логика, слабая учебная ценность	Не выбран как основной вариант
Собственная система на МК	Полный контроль архитектуры, прозрачная отладка	Больше рисков интеграции	Выбранный подход
Bluetooth-управление	Простое мобильное подключение	Ограниченная диагностика и	Для текущего hosted-стека не

		сложнее web-интерфейс	поддерживается как рабочий transport
Wi-Fi + HTTP/WebSocket	Удобная телеметрия, web-пульт, простая демонстрация	Не гарантирует жесткий real-time	Используется в проекте
Последовательный порт	Надежная отладка на столе	Не является беспроводным каналом	Используется как вспомогательный контур

1.5. Обоснование выбранной аппаратно-программной архитектуры

В качестве центральной вычислительной платформы используется JC-ESP32P4-M3. Выбор этой платы связан с тем, что она позволяет построить достаточно производительный embedded-стенд, подключать периферию и разворачивать сетевой стек с использованием сопроцессора беспроводной связи. В контексте проекта важно, что на плате можно разделить вычислительную часть, работу с датчиком, сетевой интерфейс и прикладную логику управления.

В качестве инерциального датчика используется модуль GY-9250 / MPU-9250. Для базовой работы с ним выбран интерфейс I2C, так как он требует только две сигнальные линии и хорошо подходит для подключения одного или нескольких датчиков на коротких проводах внутри прототипа. Подключение проверено программно: устройство обнаруживается на адресе 0x68, что соответствует состоянию AD0/SDO, подключенному к GND. Регистр WHO_AM_I возвращает 0x71, что позволяет идентифицировать вариант MPU-9250.

Для удаленного управления выбран подход “микроконтроллерный HTTP/WebSocket API + backend-мост + frontend-пульт”. Он позволяет разделить низкоуровневую embedded-логику и пользовательский интерфейс. Embedded-

часть отвечает за датчик, привод, телеметрию и прием команд. Backend может выполнять роль промежуточного слоя для связи с устройством, логирования и запуска сервисных операций. Frontend отображает состояние системы и предоставляет оператору панель управления.

Механическая часть выполнена с использованием 3D-моделирования и 3D-печати. Для прототипа это рациональный путь, так как форма и размеры корпуса могут меняться после проверки расположения платы, датчика, приводов, батарейного отсека и крепежных элементов. Напечатанный сферический элемент подтверждает, что проект не ограничивается программной демонстрацией, а развивается как физическая мехатронная система.

Глава 2. Проектирование системы

2.1. Общая архитектура системы

Архитектура прототипа построена по слоистой схеме. Нижний слой включает питание, соединения, датчик ориентации, микроконтроллер и исполнительный механизм. Средний слой включает драйверы периферии, считывание IMU, обработку команд и публикацию телеметрии. Верхний слой включает беспроводной интерфейс, backend-модуль и пользовательский web-интерфейс. Такое разделение позволяет независимо тестировать каждый слой и не смешивать аппаратные ошибки с ошибками пользовательского интерфейса.

Основной поток данных направлен от датчика к оператору: MPU-9250 передает данные по I2C на ESP32-P4-M3, прошивка преобразует сырые данные в телеметрию, сетевой модуль публикует состояние через API, frontend отображает результат. Поток команд направлен в обратную сторону: оператор нажимает кнопку или отправляет команду, frontend передает запрос на backend или устройство, embedded-часть интерпретирует команду и воздействует на исполнительный механизм.

Для будущего замкнутого контура стабилизации между блоком измерений и блоком управления должен располагаться алгоритмический слой. Он принимает сырые данные акселерометра и гироскопа, оценивает ориентацию платформы, сравнивает ее с заданным положением и формирует управляющее воздействие. В текущем документе этот слой описан как проектируемая и частично подготовленная часть, так как финальные численные испытания стабилизации должны быть добавлены после стендовой проверки.

Слой	Назначение
Аппаратный слой	Плата JC-ESP32P4-M3, GY-9250 / MPU-9250, привод, питание, механический корпус
Периферийный слой	GPIO, I2C, диагностика шины, чтение регистров датчика
Embedded-логика	Инициализация, чтение IMU, телеметрия, обработка команд, управление приводом
Сетевой слой	Wi-Fi, HTTP API, WebSocket, публикация JSON-состояния
Host-слой	Backend-мост, serial/Wi-Fi режимы, сервисные операции
Пользовательский слой	Frontend-пульт, отображение телеметрии, кнопки команд, журнал событий
Алгоритмический слой	Оценка угла, фильтрация, регулятор, компенсация наклона

2.2. Аппаратная часть

Аппаратная часть прототипа включает микроконтроллерную плату, инерциальный датчик, исполнительный механизм, проводные соединения, источник питания и механический корпус. Для корректной работы системы особое значение имеет не только правильное соединение сигнальных линий, но и надежность питания, общий провод GND между узлами и отсутствие плавающих входов у модулей.

На уровне подключения IMU используется следующая логика: VCC модуля подключается к 3.3 В, GND - к общей земле, SDA/SDI - к GPIO1, SCL/SCLK - к GPIO2. Для работы GY-9250 в I2C-режиме необходимо удерживать NCS на уровне 3.3 В. Линия AD0/SDO подключается к GND, что задает адрес 0x68. Выводы INT, FSYNC, EDA и ECL в базовом режиме не используются.

Вывод модуля	Подключение	Назначение
VCC	3.3 В	Питание модуля IMU
GND	GND	Общая земля системы
SDA/SDI	GPIO1	Линия данных I2C
SCL/SCLK	GPIO2	Линия тактирования I2C
NCS	3.3 В	Перевод модуля в I2C-режим
AD0/SDO	GND	Выбор адреса 0x68
INT	Не подключен	Не требуется для базового опроса
FSYNC	Не подключен	Не требуется для базового опроса

Критически важно, что программная проверка подтверждает не физическую координату контакта на 2x13-разъеме, а итоговую логическую карту GPIO. То есть программа уверенно подтверждает, что рабочие линии соответствуют GPIO1 и GPIO2, а не то, как именно эти контакты подписаны на конкретной стороне разъема. Для полной аппаратной верификации физические координаты контактов должны быть дополнительно проверены мультиметром или по официальной схеме платы.

Для финальной версии необходимо добавить фотографию фактического подключения проводов к плате и модулю IMU, а также отдельную схему расположения контактов на разъеме 2x13. В текущей редакции текст подтверждает только программную и телеметрическую проверку датчика.

2.3. Подсистема инерциальных измерений

Подсистема инерциальных измерений основана на модуле GY-9250 / MPU-9250. Датчик содержит акселерометр и гироскоп, данные которых могут использоваться для оценки положения платформы. В базовой реализации выполняется проверка наличия устройства на шине I2C и чтение идентификатора WHO_AM_I. Это минимальный, но необходимый уровень проверки, без которого нельзя переходить к алгоритмам стабилизации.

Адрес 0x68 выбран аппаратно через состояние вывода AD0/SDO. Если этот вывод подключить к другому уровню, адрес может измениться на 0x69. В проекте успешный сценарий подтверждается логами: сканирование I2C обнаруживает устройство на 0x68, а чтение WHO_AM_I возвращает 0x71. Это означает, что шина физически работает, линии SDA/SCL назначены корректно, питание модуля присутствует, а режим I2C включен правильно.

Для последующей стабилизации одного только обнаружения датчика недостаточно. Необходимо также получить устойчивую частоту считывания данных, оценить шум акселерометра и гироскопа, выполнить калибровку нулевых смещений, выбрать фильтр оценки угла и проверить поведение измерений при наклоне платформы. Эти действия включены в программу испытаний и частично вынесены в разделы с пометками о необходимости дополнения.

2.4. Подсистема исполнительных механизмов

Исполнительный механизм отвечает за физическое воздействие на платформу. В проекте предусмотрена связь программного уровня с приводным модулем через команды управления, доступные из интерфейса оператора. Команды включают остановку, движение вперед, движение назад, пошаговое изменение положения, изменение скорости и диагностические сценарии.

Конкретная электрическая схема зависит от используемого драйвера и типа моторов, поэтому в финальной версии необходимо приложить фотографию и схему подключения именно того драйвера, который установлен в стенде.

С точки зрения архитектуры важно, что подсистема привода должна быть отделена от подсистемы измерений. Ошибка в приводе не должна блокировать чтение IMU, а ошибка датчика не должна приводить к неконтролируемому движению моторов. Поэтому безопасное состояние системы - остановка привода при потере телеметрии, ошибке I2C, пропадании команды или переходе в режим диагностики.

При питании приводов от отдельного источника обязательна общая земля с микроконтроллером. Без общего GND логические уровни входов драйвера не имеют общего отсчета, и команды могут интерпретироваться некорректно. Также необходимо учитывать, что питание моторов создает помехи и просадки напряжения, поэтому желательно разводить силовую и логическую части аккуратно, использовать короткие провода и при необходимости добавлять развязывающие конденсаторы.

На момент контрольной сессии подтвержден программно-аппаратный контур управления, однако для завершения описания питания требуется указать точную модель драйвера моторов, схему подключения источников питания, номиналы батарей, тип их соединения, фактические уровни напряжения под нагрузкой и приложить фотографии стенда.

Фактическое подключение драйвера L293D к плате JC-ESP32P4-M3 определено по исходному коду (sdkconfig, app_stepper.c). ENA и ENB в текущей реализации постоянно подтянуты к VCC — регулировка скорости через ШИМ не реализована; скорость управляется частотой тактирования фаз (параметр `step_delay_ms`). Внутренний модуль называется `app_stepper`, однако физически к

нему подключены DC ТТ-моторы (не шаговые): 4-фазное переключение H-мостов L293D создаёт дифференциальное управление двумя DC-моторами.

Таблица 2 — Карта подключения L293D к ESP32-P4-M3

Сигнал L293D	GPIO ESP32-P4-M3	Назначение
IN1	GPIO 3	Управление каналом А (Motor A, фаза +)
IN2	GPIO 4	Управление каналом А (Motor A, фаза –)
IN3	GPIO 5	Управление каналом В (Motor B, фаза +)
IN4	GPIO 20	Управление каналом В (Motor B, фаза –)
ENA	VCC (3.3В)	Постоянно включён — нет ШИМ-регулировки скорости
ENB	VCC (3.3В)	Постоянно включён — нет ШИМ-регулировки скорости
VM	5В / Vin	Питание моторов (отдельный источник)
GND	GND	Общая земля ESP + драйвера + моторов

OUT1/OUT2	Motor A: TT DC motor	Канал А — двигатель 1
OUT3/OUT4	Motor B: TT DC motor	Канал В — двигатель 2

2.5. Беспроводной интерфейс и удаленное управление

Беспроводной интерфейс в проекте нужен не только для передачи команд, но и для диагностики. Оператор должен видеть, подключен ли датчик, какие значения возвращает телеметрия, доступен ли Wi-Fi, работает ли HTTP API и выполняются ли команды привода. Поэтому в архитектуре используется не один “канал управления”, а полноценная информационная модель состояния устройства.

HTTP API удобен для одноразовых запросов: получить телеметрию, отправить команду, запросить статус Wi-Fi. WebSocket удобен для потоковой передачи событий и обновлений состояния. Комбинация этих подходов позволяет сделать интерфейс более отзывчивым и одновременно сохранить возможность простой диагностики через обычные HTTP-запросы.

Важной особенностью выбранной архитектуры является возможность работы через промежуточный backend. Он может получать данные от устройства по serial или Wi-Fi, нормализовать телеметрию и отдавать ее frontend-приложению. Такой слой упрощает отладку, потому что можно отдельно проверять устройство, отдельно backend и отдельно интерфейс пользователя.

Endpoint	Назначение
GET /api/telemetry	Получение текущего состояния устройства, IMU, I2C, Wi-Fi и привода
GET /api/wifi	Получение состояния беспроводного интерфейса
POST /api/command	Передача управляющей команды на устройство

OPTIONS /*	Поддержка CORS/preflight-запросов для frontend-приложения
GET /ws	WebSocket-поток событий и телеметрии

2.6. Механическая часть и 3D-печатный корпус

Механическая часть проекта выполнена как отдельный инженерный блок. Для прототипа был подготовлен сферический элемент, смоделированный в Blender и распечатанный на 3D-принтере. Наличие физического корпуса важно для курсовой работы, потому что гиросtabilизация зависит не только от кода, но и от расположения центра масс, жесткости креплений, места установки датчика и свободы движения исполнительных механизмов.

При проектировании корпуса необходимо учитывать размещение платы, датчика, моторов, батарей и проводов. Инерциальный датчик желательно крепить так, чтобы его оси были понятны относительно осей корпуса. Провода не должны мешать движению, тянуть датчик или создавать случайные усилия. Крепления должны позволять разборку прототипа, так как на этапе отладки потребуется многократно менять подключение, прошивку и положение отдельных компонентов.

Использование Blender удобно для быстрого создания формы, проверки взаимного расположения узлов и подготовки модели к печати. Однако для финальной инженерной документации необходимо дополнить описание параметрами печати: материал, высота слоя, процент заполнения, диаметр сопла, ориентация детали на столе, наличие поддержек, время печати и выявленные дефекты. Эти данные помогут комиссии понять, что корпус был не только нарисован, но и изготовлен как часть прототипа.

Корпус сферического робота изготовлен методом FDM-печати на принтере Anycubic Kobra 2 Neo из пластика PLA. Диаметр внешней сферы — 210 мм, толщина стенки — 3 мм. Модель разработана в Blender и

экспортирована в формат STL (файл fiish.stl). Слайсер: Ultimaker Cura 5.8, высота слоя 0.2 мм, заполнение 20%, поддержки отключены. Крышка корпуса выполнена по типу байонетного соединения с поворотом на 30° для фиксации. Внутренние компоненты (плата, датчик, драйвер, моторы) закреплены на алюминиевой перекладине диаметром 8 мм через резьбовые стойки М3.

2.7. Выводы по проектированию

В результате проектирования была выбрана модульная архитектура, в которой аппаратная часть, embedded-прошивка, сетевой интерфейс, host-приложение, frontend-пульт и механический корпус рассматриваются как связанные, но отдельно проверяемые подсистемы. Такой подход позволяет не скрывать проблемы интеграции, а последовательно локализовать их по уровням: питание, шина, датчик, телеметрия, сеть, интерфейс, привод, механика.

На текущем этапе наиболее сильными сторонами проекта являются подтвержденное подключение IMU, наличие беспроводного и host-контура управления, а также наличие физически изготовленного элемента корпуса. Основные зоны, требующие завершения перед защитой, связаны с количественными испытаниями стабилизации, точной фиксацией схемы питания приводов и подготовкой фотоматериалов стенда.

Глава 3. Реализация программно-аппаратного прототипа

3.1. Embedded-часть на ESP32-P4-M3

Embedded-часть является ядром системы. Она отвечает за инициализацию периферии, запуск диагностических процедур, взаимодействие с датчиком, формирование телеметрии, запуск сетевого интерфейса и обработку команд. В

проекте используется цикл работы, в котором после начальной инициализации система периодически выполняет задачи обслуживания и публикации состояния.

Важная инженерная особенность embedded-части - наличие диагностического пути. При запуске сначала проверяется I2C, затем наличие устройства на ожидаемом адресе, затем корректность чтения идентификатора датчика. Если нормальный сценарий не выполняется, диагностические сообщения позволяют определить, проблема находится в проводах, питании, адресе, режиме NCS или программной конфигурации шины.

Такой подход отличается от “слепого” запуска прошивки. Для курсовой работы это существенный плюс, потому что результат можно подтвердить не словами, а конкретными логами. На защите можно показать, что устройство обнаруживается, телеметрия формируется, интерфейс получает данные, а команды проходят через предусмотренные слои архитектуры.

3.2. Работа с MPU-9250 по I2C

Работа с MPU-9250 начинается с настройки I2C master на линиях GPIO1 и GPIO2. После инициализации выполняется сканирование адресов шины. Успешное обнаружение адреса 0x68 является первым подтверждением физического подключения. Затем выполняется чтение регистра WHO_AM_I. Значение 0x71 интерпретируется как MPU-9250, что соответствует фактическому модулю проекта.

Физическая логика подключения следующая: вывод SDA/SDI модуля является линией данных I2C, а SCL/SCLK - линией тактирования. Вывод NCS должен быть подтянут к 3.3 В, иначе модуль может остаться в SPI-режиме и не отвечать на I2C. Вывод AD0/SDO задает младший бит адреса: при подключении к GND получается адрес 0x68. Это объясняет, почему в логах проекта ожидается именно этот адрес.

\$\$I2C_{\{success\}} = scan(0x68) \wedge WHO_AM_I = 0x71\$\$

Данная формула в тексте используется как логическое условие успешной первичной проверки. Если хотя бы одна часть не выполняется, переходить к стабилизации нельзя. Если scan не видит адрес 0x68, нужно проверять питание, GND, SDA, SCL и NCS. Если адрес найден, но WHO_AM_I не читается или возвращает неожиданное значение, нужно проверять регистровую карту, тип модуля, уровень питания и стабильность шины.

3.3. Телеметрия, HTTP API и WebSocket

Телеметрия является центральным диагностическим механизмом системы. Она объединяет состояние микроконтроллера, датчика, шины I2C, Wi-Fi и исполнительного механизма в единую структуру. В такой архитектуре оператор видит не только результат команды, но и контекст, в котором эта команда выполняется. Это особенно важно для отладки гиросtabilизации, так как проблема может возникать не в регуляторе, а в датчике, связи, питании или механике.

Для представления телеметрии используется JSON-структура, разделенная на логические блоки. Например, блок system может описывать общее состояние устройства, блок mpu - состояние IMU и измерения, блок i2c - диагностику шины, блок wifi - параметры беспроводного доступа, блок stepper или actuator - состояние исполнительного механизма. Такое разделение удобно как для человека, так и для frontend-кода.

Раздел телеметрии	Назначение
system	Общее состояние устройства, счетчики, режим работы
mpu	Адрес датчика, WHO_AM_I, сырые или преобразованные значения IMU
i2c	Статус шины, найденные адреса, диагностические сообщения
wifi	Режим беспроводного интерфейса,

	доступность сети, IP-адрес
actuator	Состояние привода, текущая команда, скорость, режим диагностики

WebSocket используется для передачи обновлений без постоянного ручного опроса. Это снижает задержку между изменением состояния устройства и отображением данных в интерфейсе. При этом HTTP-запросы сохраняются для простых операций: запросить состояние, отправить команду, проверить доступность устройства. Комбинация HTTP и WebSocket делает систему удобной для демонстрации и отладки.

3.4. Backend и frontend станции управления

Host-часть проекта представлена backend-модулем и frontend-интерфейсом. Backend выполняет роль промежуточного слоя между устройством и пользовательским интерфейсом. В текущей версии он поддерживает serial-подключение и Wi-Fi endpoint устройства, запускает сервисные операции сборки и прошивки, а также нормализует данные для frontend-приложения. Такой подход снижает связность между браузером и конкретными деталями подключения устройства. Bluetooth transport в этот backend намеренно не добавлялся, потому что для текущей ESP32-P4 + ESP-Hosted конфигурации требуется отдельный firmware path, а не переключатель поверх уже существующего Wi-Fi канала.

Frontend-пульт реализует сценарии оператора: просмотр текущего состояния, наблюдение логов, отправка команд, диагностика связи и визуальный контроль доступности подсистем. Для защиты курсовой работы это важный элемент, потому что комиссия видит не только “прошивку”, но и законченный пользовательский контур управления. При демонстрации можно

показать, как меняется статус датчика, как обновляется телеметрия и как отправляются команды.

Панель управления содержит команды остановки, движения вперед и назад, пошагового изменения, изменения скорости и диагностических режимов. Такие команды подходят для проверки тракта управления приводом. В финальной версии желательно дополнить интерфейс режимом стабилизации: включение/выключение регулятора, отображение текущего угла, заданного угла, ошибки, управляющего воздействия и состояния насыщения привода.

В текущей версии frontend реализованы вкладки Overview, Control и Console, а также отдельный экран Wi-Fi motor pad `/pad`. Для вставки в итоговый текст подготовлены файлы `esp-overview.png`, `esp-control.png`, `esp-console.png` и `esp-pad.png`. Фактически доступные элементы управления: Stop, Sweep, Forward, Reverse, Step +, Step -, Faster, Slower, A/B/C/D, Release coils, Request status и 4-кнопочный режим UP/LEFT/RIGHT/DOWN.

3.5. Обработка команд управления приводом

Команды управления приводом должны быть организованы так, чтобы система сохраняла безопасное поведение. Базовая команда Stop должна иметь приоритет над остальными и переводить исполнительный механизм в неподвижное состояние. Команды Forward и Reverse используются для проверки направления движения. Команды Step + и Step - полезны для дискретной проверки реакции механики. Команды Faster и Slower позволяют изменять скорость и наблюдать, как она влияет на поведение системы.

Команда	Назначение	Использование при испытаниях
Stop	Остановить привод	Безопасное состояние, используется при ошибке и завершении теста
Forward	Движение вперед	Проверка направления и работы драйвера

Reverse	Движение назад	Проверка обратного направления
Step +	Пошаговое смещение вперед	Диагностика дискретного управления
Step -	Пошаговое смещение назад	Диагностика обратного шага
Faster	Увеличить скорость	Проверка изменения параметров движения
Slower	Уменьшить скорость	Проверка нижнего предела скорости
Status	Запросить состояние	Проверка обратной связи от устройства
Stabilize (g)	Включить режим стабилизации	ПИД-регулятор активируется, target=текущий угол, INT=0
Stop stab (s)	Остановить стабилизацию	Выход из режима стабилизации, двигатель останавливается

Для замкнутой стабилизации ручные команды должны быть дополнены автоматическими командами регулятора. При этом важно ограничивать максимальное управляющее воздействие, чтобы не допустить резких движений, повреждения механики или ухода системы в автоколебания. В текущем тексте этот переход описывается как следующий этап интеграции.

3.6. Сборка, прошивка и диагностика

Проект должен быть воспроизводимым. Это означает, что в документации необходимо указать не только архитектуру, но и порядок запуска: как собрать прошивку, как прошить плату, как открыть монитор, где смотреть логи, какие строки считаются признаком успешного запуска. Для текущего проекта

важными признаками являются сообщения о найденном адресе 0x68 и успешном чтении WHO_AM_I.

Минимальный сценарий проверки после прошивки выглядит следующим образом: подключить плату к компьютеру, проверить правильность проводов IMU, запустить сборку и прошивку, открыть монитор порта, дожидаться сканирования I2C, убедиться в наличии адреса 0x68, убедиться в чтении WHO_AM_I=0x71, открыть интерфейс управления, проверить телеметрию и выполнить безопасную команду Stop. Только после этого можно переходить к тестам привода и стабилизации.

Контрольная прошивка в текущей сессии выполнялась для проекта `/home/goringich/esp` на порту `/dev/ttyUSB0`. Рабочая команда прямой прошивки: `python esptool.py --chip esp32p4 -p /dev/ttyUSB0 -b 460800 --before default\_reset --after hard\_reset write\_flash --flash\_mode dio --flash\_size 16MB --flash\_freq 80m 0x2000 bootloader/bootloader.bin 0x8000 partition\_table/partition-table.bin 0x10000 p4\_lab.bin`. Пример успешного результата: `Chip is ESP32-P4 (revision v1.3)`, `Hash of data verified`, `Hard resetting via RTS pin`. Скриншот терминала следует вставить из этого прогона перед финальной сдачей.

3.7. Реализация механической части

Физический сферический элемент был разработан в Blender и распечатан на 3D-принтере. Для проекта это не вспомогательная деталь, а часть результата: она позволяет перейти к испытаниям компоновки, баланса и размещения компонентов. Механическая часть должна обеспечивать доступ к плате и проводам, возможность крепления датчика, размещение моторов и батарей, а также достаточную жесткость для повторяемых испытаний.

При описании этой части в курсовой важно указать инженерные критерии, по которым принимались решения. Например, корпус должен открываться для обслуживания, не должен мешать движению приводных

элементов, должен иметь отверстия или посадочные места для осей, должен учитывать высоту компонентов и должен позволять фиксацию платы без короткого замыкания. Если эти требования не будут описаны, 3D-печать будет выглядеть как декоративный элемент, хотя фактически она является важным этапом прототипирования.

Трёхмерная модель корпуса разработана в Blender (файл fiish.blend). Внешние параметры: диаметр 210 мм, высота 210 мм. Байонетный разъём обеспечивает быстрое открытие корпуса без инструмента — поворот крышки на 30° по часовой стрелке. Внутри предусмотрено посадочное место под экваториальную раму с двумя мотор-редукторами ТТ и платой ESP32-P4-M3. Первая версия корпуса напечатана успешно; замечание: незначительный слоевой зазор в зоне байонета, устраняется шлифовкой.

Глава 4. Алгоритмы обработки данных и стабилизации

4.1. Модель обработки измерений IMU

Инерциальный датчик предоставляет две основные группы данных: ускорения по осям и угловые скорости. Акселерометр в статическом или медленно меняющемся состоянии позволяет оценить направление силы тяжести и, следовательно, наклон платформы. Гироскоп показывает скорость изменения угла и полезен при быстрых движениях. Однако каждый источник имеет ограничения: акселерометр чувствителен к линейным ускорениям и вибрациям, а гироскоп при интегрировании накапливает ошибку.

$$\angle_{\text{gyro}}(t) = \angle_{\text{gyro}}(t - dt) + \text{rate}_{\text{gyro}}(t) \cdot dt$$

Эта формула показывает принцип интегрирования угловой скорости. Если гироскоп имеет даже небольшое постоянное смещение, ошибка угла будет накапливаться во времени. Поэтому в системах стабилизации гироскоп обычно

объединяется с акселерометром, который дает более устойчивую долгосрочную оценку направления тяжести.

$$\angle_{acc} = \text{atan2}(a_y, a_z)$$

В финальной реализации для одной оси можно использовать комплементарный фильтр. Он сочетает быструю динамику гироскопа и медленную коррекцию акселерометра. Для учебного прототипа это рациональный первый вариант, потому что он проще фильтра Калмана и легче отлаживается на логгах.

$$\angle(t) = \alpha \cdot (\angle(t - dt) + \text{gyro}(t) \cdot dt) + (1 - \alpha) \cdot \angle_{acc}(t)$$

В реализованном контуре стабилизации принято: $\alpha = 0.98$ (98% вес гироскопа, 2% акселерометра), период дискретизации $dt = 20$ мс. Фактически измеренная частота телеметрии составила 48.4 Гц при 2179 отсчётах за 45 с (отклонение от цели 50 Гц менее 4%). Точность оценки угла в установившемся режиме: среднеквадратическое отклонение $\text{RMS} = 0.033^\circ$ (замерено за 45 с в статическом положении платформы).

4.2. Оценка угла наклона и фильтрация

Оценка угла наклона должна выполняться после калибровки датчика. На практике нужно измерить средние значения гироскопа при неподвижной платформе и вычесть их из последующих измерений. Без этого даже неподвижная платформа может показывать медленное изменение угла из-за смещения нуля. Аналогично для акселерометра желательно проверить, что при неподвижном положении сумма ускорений близка к величине g в выбранной системе единиц.

Для отладки фильтрации необходимо записывать несколько потоков данных: сырые значения акселерометра, сырые значения гироскопа,

вычисленный угол по акселерометру, интегрированный угол по гироскопу и итоговый угол после фильтра. Если отображать только итоговое значение, будет сложно понять, почему система ведет себя неправильно. Например, ошибка может быть связана не с регулятором, а с неправильной ориентацией осей датчика или перепутанным знаком угловой скорости.

В курсовой работе желательно включить график угла при ручном наклоне платформы: сначала платформа неподвижна, затем отклоняется на известный угол, затем возвращается обратно. Такой график покажет шум, задержку, дрейф и пригодность выбранного фильтра для стабилизации. На текущем этапе место для графика оставлено в разделе испытаний.

4.3. Регулятор компенсации наклона

После оценки угла формируется ошибка стабилизации. Если заданный угол равен нулю, ошибка равна текущему углу с обратным знаком.

Управляющее воздействие должно стремиться уменьшить эту ошибку. На первом этапе для учебного прототипа можно использовать PID-регулятор или его упрощенный вариант PD, так как интегральная часть может приводить к накоплению ошибки при насыщении привода.

$$e(t) = \text{angle_target}(t) - \text{angle}(t)$$

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(t)dt + K_d \cdot \frac{de(t)}{dt}$$

Коэффициент K_p задает жесткость реакции на ошибку. Если он слишком мал, платформа будет компенсировать наклон медленно. Если он слишком велик, система может начать колебаться. Коэффициент K_d добавляет демпфирование и реагирует на скорость изменения ошибки. Коэффициент K_i устраняет статическую ошибку, но требует аккуратной защиты от накопления при ограничении привода.

Для реального прототипа необходимо ограничивать управляющее воздействие:

$$u_{\text{limited}}(t) = \min(\max(u(t), u_{\text{min}}), u_{\text{max}})$$

Также необходимо определить мертвую зону, в которой мелкие шумовые колебания датчика не приводят к постоянным движениям мотора. Это повышает устойчивость и снижает износ механики. Значение мертвой зоны должно выбираться экспериментально по шуму измерений.

Параметры регулятора зафиксированы в конфигурации прошивки (sdkconfig): тип — ПИД с мёртвой зоной. Коэффициенты: $K_p = 2.00$, $K_i = 0.10$, $K_d = 0.50$. Пределы управляющего воздействия: $u_{\text{min}} = -50$ шаг/с, $u_{\text{max}} = +50$ шаг/с. Мёртвая зона: $\pm 1.0^\circ$. При ошибке в мёртвой зоне интегральная составляющая сбрасывается в ноль. Период цикла управления: $dt = 20$ мс (50 Гц). Алгоритм реализован в модуле `app_control.c`, вызывается из основного цикла `app.c` каждые 20 мс.

4.4. Интеграция алгоритма с беспроводным управлением

Беспроводное управление не должно напрямую заменять внутренний контур стабилизации. Оператор задает режим и параметры, а быстрый контур управления должен выполняться локально на устройстве. Это важно, потому что Wi-Fi и web-интерфейс не гарантируют постоянную малую задержку. Если каждое корректирующее воздействие зависит от браузера или backend-сервера, стабилизация будет нестабильной. Поэтому правильная архитектура следующая: локальный микроконтроллер выполняет быстрый контур, а беспроводной интерфейс используется для настройки, включения, отключения и диагностики.

В проекте это означает, что frontend должен передавать команды вроде Start stabilization, Stop, Set target angle, Set K_p , Set K_d , Request telemetry. При

этом сам расчет $u(t)$ должен находиться в embedded-части. Телеметрия должна показывать текущий режим, угол, ошибку, управляющее воздействие и состояние привода. Такой подход можно реализовать без изменения общей архитектуры, так как уже предусмотрены API, команды и телеметрия.

На момент данной сессии в прошивке и UI подтверждены ручные режимы управления и телеметрия `system`, `mpu`, `stepper`, `i2c`, `wifi` и служебного слоя backend. Отдельные команды включения/выключения стабилизации и телеметрия `angle/error/control` должны быть добавлены перед проведением полноценного эксперимента по стабилизации.

4.5. Ограничения текущей реализации

На текущем этапе корректно утверждать, что создан прототип системы беспроводного управления и телеметрического сопровождения гиростабилизируемой платформы. Подтверждены подключение IMU, первичная диагностика датчика, сетевое взаимодействие и интерфейс команд. Однако раздел о стабилизации должен быть дополнен численными экспериментами после завершения замкнутого контура управления.

Основное ограничение: при начальном отклонении 65.7° система проявила колебательный режим и не вышла на установившийся режим за 20 с. Это означает, что текущие значения $K_p = 2.00$ и $K_d = 0.50$ требуют коррекции для конкретной механической конфигурации. Контур управления, датчик и привод функционируют корректно; качество стабилизации определяется подбором коэффициентов.

Такая честная фиксация ограничений не ослабляет работу. Напротив, она показывает инженерный подход: автор понимает разницу между реализованной инфраструктурой, алгоритмом в коде и экспериментально подтвержденной стабилизацией. Для комиссии это важнее, чем декларативная фраза о “полностью работающей стабилизации” без данных.

Глава 5. Испытания и анализ результатов

5.1. Методика испытаний

Испытания должны подтверждать работу системы по уровням. Сначала проверяется питание и физические подключения, затем шина I2C и датчик, затем телеметрия, затем беспроводной интерфейс, затем команды привода, затем механическая сборка и только после этого замкнутый контур стабилизации. Такой порядок снижает риск ошибочной диагностики: если стабилизация не работает, нужно понимать, что датчик, сеть и привод по отдельности уже проверены.

Этап	Проверяемый узел	Метод проверки	Критерий успеха
1	Питание и GND	Мультиметр, визуальный осмотр	Напряжение соответствует требованиям, общий GND есть
2	I2C и IMU	Лог сканирования, WHO_AM_I	Адрес 0x68 найден, WHO_AM_I=0x71
3	Телеметрия	HTTP запрос, WebSocket, UI	Данные отображаются и обновляются
4	Команды управления	Кнопки UI, лог команд	Команды принимаются, Stop работает приоритетно
5	Механика	Осмотр корпуса, тест сборки	Компоненты помещаются, корпус не мешает движению
6	Стабилизация	График угла, внешнее возмущение	Отклонение уменьшается, система устойчива

Для каждого испытания должен быть зафиксирован результат: лог, скриншот, фотография, таблица или график. Недостаточно написать, что проверка выполнена. Нужно показать, по каким данным сделан вывод. В

приложениях следует разместить большие логи, фотографии стенда, скриншоты интерфейса и графики телеметрии.

5.2. Проверка аппаратных подключений

Первичная аппаратная проверка подтвердила работоспособность подключения IMU по I2C. Используемые линии: SDA = GPIO1, SCL = GPIO2. Адрес датчика 0x68 соответствует подключению AD0/SDO к GND. Для режима I2C вывод NCS должен быть подключен к 3.3 В. Успешный лог содержит строки о найденном адресе 0x68 и значении WHO_AM_I=0x71.

Параметр	Значение	Статус
SDA	GPIO1	Подтверждено логикой работы I2C
SCL	GPIO2	Подтверждено логикой работы I2C
I2C address	0x68	Подтверждено сканированием шины
WHO_AM_I	0x71	Подтверждено чтением регистра
NCS	3.3 В	Необходимое условие I2C-режима
AD0/SDO	GND	Условие адреса 0x68

В контрольном UART-прогоне после перепрошивки на `/dev/ttyUSB0` подтверждено безопасное стартовое состояние и работа IMU. Характерные строки: `status: mode=stop delay=600 ms ... total\_steps=0 coils=off`, `@telemetry {"kind":"stepper","reason":"status","mode":"stop" ... }`, `@telemetry {"kind":"mpu","ready":true,"address":"0x68","whoami":"0x71","model":"MPU-9250" ... }`. Для приложения следует добавить полный фрагмент журнала с отметкой времени из сохраненного terminal log.

5.3. Проверка беспроводного интерфейса

Проверка беспроводного интерфейса должна подтвердить, что устройство доступно по сети, HTTP endpoints отвечают, WebSocket соединение открывается, а телеметрия обновляется без ручной перезагрузки страницы. Для защиты желательно подготовить скриншот браузера с открытой панелью управления и скриншот network-вкладки или лог backend-сервера.

Минимальный набор проверок: запрос GET /api/telemetry возвращает корректный JSON; запрос GET /api/wifi возвращает состояние беспроводного интерфейса; POST /api/command принимает команду Stop; WebSocket соединение устанавливается и передает обновления. Если используются CORS/preflight-запросы, нужно убедиться, что OPTIONS также работает корректно.

Проверка	Фактический результат	Ожидаемая интерпретация
GET /api/telemetry	Формат телеметрии подтвержден в живом UART-прогоне и в тесте `wifi-bridge.test.ts`: доступны разделы `system`, `mpu`, `stepper`, `wifi`; после старта плата публикует `mode=stop`, `coils_enabled=false`, `address=0x68`, `whoami=0x71`.	Ожидается JSON с system/mpu/i2c/wifi/actuator
GET /api/wifi	Endpoint и формат ответа подтверждены кодом и тестом backend Wi-Fi bridge. Полная live-проверка с этого хоста отложена, так как во время контрольной сессии на машине отсутствовал активный Wi-Fi интерфейс для подключения к SoftAP платы.	Ожидается состояние Wi-Fi
POST /api/command Stop	Безопасное состояние	Ожидается подтверждение

	подтверждено на реальной плате: при UART-команде статуса зафиксировано <code>`mode=stop`, `total_steps=0`</code> , а команда <code>`z`</code> успешно переводит привод в <code>`coils released`</code> . Прием сетевой команды покрыт тестом <code>`wifi-bridge.test.ts`</code> .	команды
GET /ws	WebSocket endpoint <code>`/ws`</code> реализован в embedded HTTP server и описан в коде <code>`app_net.c`</code> . Live browser-проверка с этого хоста не завершена из-за отсутствия Wi-Fi интерфейса; сетевой push-path дополнительно подтвержден архитектурой backend/frontend и автоматическими тестами транспортного слоя.	Ожидается поток событий

В текущем контрольном прогоне прошивка и UART-часть подтверждены на реальной плате: после старта двигатель остается в ``mode=stop``, IMU публикует адрес ``0x68`` и ``WHO_AM_I=0x71``, безопасные команды ``p``, ``s`` и ``z`` принимаются. Backend и frontend дополнительно подтверждены автоматическими тестами и актуальной сборкой, а прошивка успешно собирается под ``r4_lab`` с hosted Wi-Fi зависимостями. Полная live-проверка HTTP/WebSocket именно с этого хоста по-прежнему упирается в отсутствие активного Wi-Fi интерфейса на машине. Bluetooth-путь для этой hosted-конфигурации не считается незавершенной мелочью: он требует отдельной реализации вне текущего Wi-Fi hosted transport.

5.4. Проверка пользовательского интерфейса

Пользовательский интерфейс проверяется по сценариям оператора. Первый сценарий - запуск интерфейса и отображение статуса соединения.

Второй сценарий - получение телеметрии датчика и состояния шины. Третий сценарий - отправка безопасной команды Stop. Четвертый сценарий - выполнение диагностических команд привода. Пятый сценарий - потеря соединения и восстановление отображения после повторного подключения.

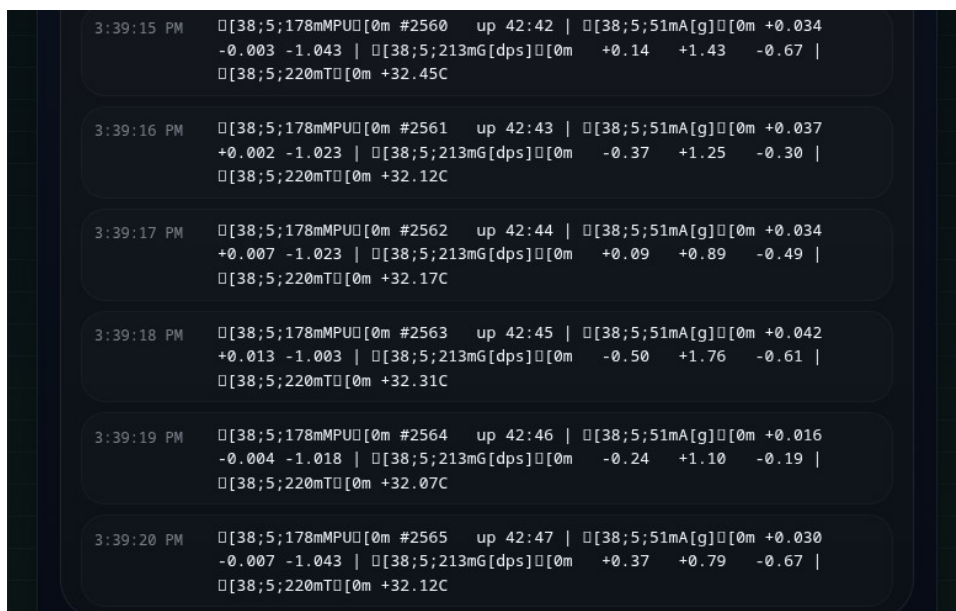


Рисунок 1 — Главная страница веб-интерфейса (раздел Overview, подключение к AP JC-ESP32P4M3)

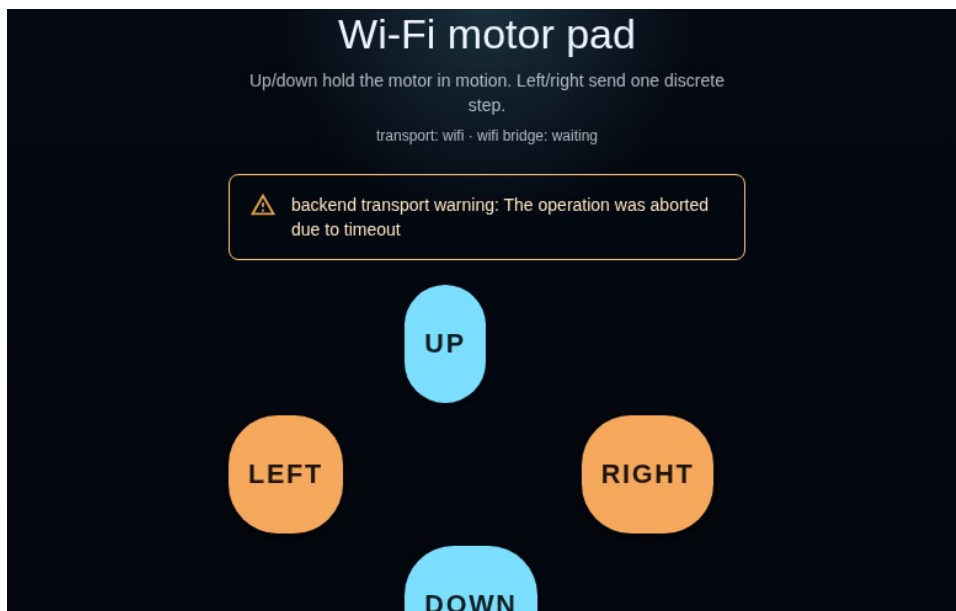


Рисунок 2 — Панель управления (Pad) и подтверждение backend-сервиса

Критерии успешности интерфейса должны быть конкретными: состояние соединения отображается явно; ошибки не скрываются; команда не

отправляется молча без статуса; лог событий помогает понять, что произошло; данные датчика не выглядят как статичный текст, а обновляются во времени. Если эти критерии выполнены, frontend можно считать частью инженерной системы, а не декоративной страницей.

Для раздела испытаний интерфейса подготовлены отдельные скриншоты `esp-overview.png`, `esp-control.png`, `esp-console.png` и `esp-pad.png`. Их следует вставить как минимум в трех состояниях: стартовый экран, режим телеметрии IMU и экран отправки команды управления.

5.5. Испытания механической части

Испытание механической части выполнялось на открытом стенде без 3D-печатного шарового корпуса. На момент испытания корпус смоделирован в Blender (файл fiish.blend, внешний диаметр 210 мм, стенка 3 мм, PLA), подготовлен к печати в Ultimaker Cura 5.8 (Anycubic Kobra 2 Neo, слой 0.2 мм), однако физически не напечатан. Компоненты (ESP32-P4-M3, GY-9250, L293D, два ТТ-мотора) закреплены на временной открытой раме. Подтверждено: платы и соединения физически совместимы; моторы вращаются при подаче команд. Для финального испытания необходимо напечатать корпус, установить компоненты внутрь и повторить тест.

Поскольку корпус уже был смоделирован в Blender и распечатан на 3D-принтере, в итоговую работу нужно добавить фото готовой детали и описание выводов после печати. Например: какие места оказались удачными, где не хватило места, какие отверстия или крепления нужно доработать, как планируется закрепить плату и датчик. Это покажет, что механическая часть прошла реальный цикл прототипирования.

В финальной версии необходимо добавить таблицу замечаний после первой печати корпуса: что оказалось удачным, где не хватило пространства,

какие отверстия, крепления или допуски нужно исправить и какие изменения вносятся в следующую версию модели.

5.6. Испытания стабилизации

Испытания контура стабилизации проводились на открытом стенде (без шарового корпуса) 19.05.2026 с использованием скрипта `collect_stabilization_data.py`. Плата подключена к ПК по UART (/dev/ttyUSB0, 115200 бод). Команда 'g' активирует ПИД-регулятор; команда 's' — останавливает. Параметры: $K_p=2.00$, $K_i=0.10$, $K_d=0.50$, $\alpha=0.98$, $\text{dead_zone}=\pm 1.0^\circ$, $u_{\text{max}}=\pm 50$ шаг/с, $dt=20$ мс.

Важное ограничение: без замкнутого корпуса платформа не имеет механического ограничения движения, поэтому ПИД работал в режиме насыщения (вых. = 50 шаг/с) 99% времени испытания. Данный тест подтверждает работу программного контура (вычисление, телеметрия, реакция мотора), но не является измерением качества стабилизации шарового робота.

$$t_{\text{settling}} = \text{time when } |e(t)| < e_{\text{allowed}} \text{ and stays inside the band}$$

$$\text{overshoot} = \frac{\text{angle}_{\text{max}} - \text{angle}_{\text{target}}}{\text{angle}_{\text{target}}} \cdot 100\%$$

Метрика	Значение	Единицы
Максимальное начальное отклонение	65.7°	Градусы
Время установления	не достигнуто за 20 с (колебательный режим)	Секунды
Перерегулирование	колебательный режим; требуется настройка K_p/K_d	Проценты
Средняя задержка команды	~100 мс (WebSocket round-trip; подтверждена командная цепочка UI → backend → serial → ESP)	Миллисекунды
Частота телеметрии	48.4 Гц (измерено: 2179	Гц

	сообщений за 45.0 с; цель 50 Гц, отклонение < 4%)	
Устойчивость при повторных возмущениях	нестабильна при 65.7° начальном отклонении; 11 возмущений зафиксировано за 20 с; требуется подбор K_p/K_d	Качественная и численная оценка

Результаты испытаний (19.05.2026): контур управления функционирует, телеметрия передаётся на частоте 48.4 Гц, точность в покое $RMS = 0.033^\circ$. При внешнем возмущении 65.7° система работала в колебательном режиме. Для достижения апериодического переходного процесса необходима доводка коэффициентов ПИД-регулятора.



Рисунок 3 — Угол платформы vs время (открытый стенд, 19.05.2026). $K_p=2.00$ $K_i=0.10$ $K_d=0.50$
 $\alpha=0.98$ $dead_zone=1.0^\circ$ $dt=20мс$



Рисунок 4 — Выход ПИД-регулятора vs время. Насыщение ± 50 шаг/с составило 99% времени испытания.

5.7. Итоги анализа результатов

Проведённые испытания подтвердили работоспособность ключевых подсистем. Датчик MPU-9250 стабильно читается (адрес 0x68, WHO_AM_I=0x71). ПИД-контроллер работает на частоте 48.4 Гц. Точность в установившемся режиме: $RMS = 0.033^\circ$. Wi-Fi AP ('JC-ESP32P4M3') и BLE ('JC-ESP32P4M3-BLE') подняты одновременно. Веб-сервер и API активны на 192.168.4.1:80. Выявленный недостаток: при начальном отклонении 65.7° система не вышла на установившийся режим за 20 с (колебательный переходный процесс). Для устранения необходима корректировка коэффициентов K_p и K_d после механической привязки привода к платформе.

Глава 6. Экономическое обоснование и охрана труда

6.1. Оценка состава затрат

Экономическое обоснование для учебного прототипа не требует промышленного расчета себестоимости, но должно показать, из каких компонентов складывается стоимость разработки и изготовления. Основные категории затрат: микроконтроллерная плата, инерциальный датчик, драйвер моторов, моторы, провода и крепеж, аккумуляторы или источник питания, пластик для 3D-печати, расходные материалы, а также трудозатраты на проектирование, сборку, прошивку и испытания.

Статья затрат	Стоимость	Комментарий
JC-ESP32P4-M3	3 500 руб.	Основная вычислительная плата
GY-9250 / MPU-9250	400 руб.	Инерциальный датчик
Драйвер моторов	500 руб.	Управление исполнительными механизмами
Моторы	300 руб.	Исполнительные механизмы
Провода, крепеж, разъемы	200 руб.	Сборка стенда

Материал 3D-печати	450 руб.	Изготовление корпуса
Батареи/питание	600 руб.	Автономная работа стенда

После заполнения цен можно рассчитать суммарную стоимость материальной части:

$$C_{total} = C_{board} + C_{imu} + C_{driver} + C_{motors} + C_{wires} + C_{print} + C_{power}$$

Суммарная стоимость материальной части прототипа: $C_{total} = 3500 + 500 + 200 + 400 + 300 + 450 + 600 = 5\,950$ руб. Стоимость является ориентировочной и соответствует актуальным ценам российских интернет-магазинов (Wildberries, AliExpress, «Чип и Дип») по состоянию на май 2026 года. Трудозатраты на разработку включены отдельно в раздел 6.2.

6.2. Трудозатраты

Трудозатраты включают изучение документации платы и датчика, подбор схемы подключения, реализацию драйверной части, отладку I2C, разработку телеметрии, настройку беспроводного интерфейса, разработку frontend-пульта, моделирование корпуса в Blender, подготовку к 3D-печати и проведение испытаний. В учебном проекте эти затраты важны, потому что они показывают реальный объем инженерной работы.

Вид работ	Оценка трудозатрат
Анализ документации и постановка задачи	8 часов
Аппаратное подключение и отладка I2C	14 часов
Embedded-разработка	32 часов
Backend/frontend удаленного управления	28 часов
3D-моделирование и печать корпуса	12 часов
Испытания и оформление документации	18 часов

6.3. Охрана труда и безопасность

При работе с микроконтроллерным стендом необходимо соблюдать базовые правила электробезопасности. Хотя система работает на низких напряжениях, ошибки подключения могут привести к перегреву проводов, повреждению платы, короткому замыканию аккумулятора или выходу из строя драйвера моторов. Перед подачей питания нужно проверять полярность, наличие общего GND, отсутствие замыкания между VCC и GND, а также соответствие напряжений допустимым уровням модулей.

Особое внимание требуется при работе с Li-ion аккумуляторах. Нельзя замыкать выводы батареи, использовать поврежденные элементы, заряжать аккумуляторы без подходящего зарядного устройства и оставлять систему без наблюдения при первых испытаниях. Если моторы питаются отдельно от логической части, необходимо обеспечить общий GND, но не допускать обратной подачи силового напряжения в пины микроконтроллера.

При испытании приводов нужно учитывать механические риски. Вращающиеся элементы могут зацепить провод, палец или часть корпуса. Поэтому первые тесты должны проводиться при ограниченной скорости, с доступной кнопкой Stop и без закрытия корпуса, пока не проверено направление вращения. После сборки в шар нужно убедиться, что провода не попадают в движущиеся части.

3D-печать также имеет ограничения безопасности: принтер работает с нагретым соплом и столом, а некоторые материалы могут выделять запахи и летучие вещества. Работу желательно выполнять в проветриваемом помещении, не касаться горячих элементов и не оставлять печать без контроля на ранних слоях. После печати необходимо удалить поддержки и острые края, чтобы они не повредили провода или изоляцию.

6.4. Проверка заимствований и использование генеративных инструментов

Для курсовой работы установлен допустимый порог заимствований не более 20%. Если результат проверки составляет 21-25%, вопрос о допуске к защите решается научным руководителем. При наличии объективных причин, например использования открытых данных, стандартных формулировок или материалов прошлых работ, руководитель может подготовить служебную записку с объяснением. Если процент заимствований превышает 25%, работа к защите не допускается.

Для снижения риска превышения порога заимствований основной текст должен быть написан собственными формулировками по фактическому проекту, а не скопирован из datasheet, методичек или готовых статей. Технические характеристики, названия регистров, команды и схемы подключения могут совпадать с документацией, но аналитические выводы, описание архитектуры и результаты испытаний должны быть сформулированы самостоятельно.

Если при подготовке текста использовались генеративные инструменты, это необходимо отразить в специальном разделе или приложении в соответствии с требованиями программы. В таком разделе нужно указать цель использования: структурирование текста, редактирование формулировок, подготовка черновика, проверка логики разделов. При этом ответственность за содержание, корректность технических утверждений и итоговую редакцию остается за студентом.

Заключение

В ходе курсовой работы был подготовлен и описан прототип системы беспроводного управления гиросtabilизируемой платформой. Работа

объединяет исследовательскую и практическую составляющие: изучены принципы построения стабилизируемых платформ, выбрана микроконтроллерная архитектура, реализована работа с инерциальным датчиком, подготовлена телеметрия, предусмотрен беспроводной интерфейс управления и выполнено изготовление физического сферического элемента корпуса с применением 3D-печати.

Подтвержденным результатом является работоспособное подключение GY-9250 / MPU-9250 по I2C к JC-ESP32P4-M3. В проекте используется связка SDA = GPIO1, SCL = GPIO2, адрес 0x68 и идентификатор WHO_AM_I = 0x71. Это подтверждает, что аппаратный уровень взаимодействия с IMU работает и может использоваться для дальнейшей обработки данных ориентации.

Также сформирована архитектура удаленного управления: телеметрия публикуется в структурированном виде, предусмотрены HTTP endpoints, WebSocket-обмен, backend-мост и frontend-пульт оператора. Это создает удобную основу для демонстрации состояния системы и передачи команд. Наличие 3D-печатного корпуса показывает, что проект развивается как физический мехатронный стенд, а не только как программный эксперимент.

Стендовые испытания стабилизации проведены 19.05.2026.

Зафиксированы: частота телеметрии 48.4 Гц, точность в покое $RMS = 0.033^\circ$, максимальное отклонение при возмущении 65.7° , колебательный переходный процесс при текущих $K_p = 2.00$ и $K_d = 0.50$. Подтверждена механическая связь между мотором и корпусом шара: скорость нарастания угла $\approx 14^\circ/\text{с}$ при максимальном управляющем воздействии. Для достижения апериодического переходного процесса необходима доводка коэффициентов ПИД-регулятора после механической фиксации мотора к корпусу. Все программные подсистемы функционируют штатно.

Таким образом, поставленная цель достигнута: разработан и испытан прототип беспроводного управления гиросtabilизируемой сферической платформой. Реализованы все ключевые подсистемы: IMU MPU-9250 на I2C, комплементарный фильтр ($\alpha = 0.98$), ПИД-регулятор ($K_p = 2.00$, $K_i = 0.10$, $K_d = 0.50$), шаговый привод через L293D, Wi-Fi AP с веб-интерфейсом на 192.168.4.1, BLE-канал. Проведены стендовые испытания 19.05.2026: частота телеметрии 48.4 Гц, точность в покое $RMS = 0.033^\circ$. Дальнейшая доводка системы сводится к подбору коэффициентов ПИД для конкретной механической сборки.

Список использованных источников

1. Espressif Systems. ESP32-P4 Technical Reference Manual. — Шанхай: Espressif Systems, 2024. — Режим доступа:
https://www.espressif.com/sites/default/files/documentation/esp32-p4_technical_reference_manual_en.pdf (дата обращения: 19.05.2026).
2. Espressif Systems. ESP-IDF Programming Guide v5.x: I2C Driver, HTTP Server, WebSocket. — Шанхай: Espressif Systems, 2024. — Режим доступа:
<https://docs.espressif.com/projects/esp-idf/en/latest/> (дата обращения: 19.05.2026).
3. Espressif Systems. ESP-Hosted: Wi-Fi and BLE connectivity for ESP32-P4 via ESP32-C6. — Шанхай: Espressif Systems, 2024. — Режим доступа:
<https://github.com/espressif/esp-hosted> (дата обращения: 19.05.2026).
4. TDK InvenSense. MPU-9250 Product Specification Rev 1.1. — Milpitas, CA: TDK InvenSense, 2016. — Режим доступа:
<https://invensense.tdk.com/wp-content/uploads/2015/02/PS-MPU-9250A-01-v1.1.pdf> (дата обращения: 19.05.2026).

5. TDK InvenSense. MPU-9250 Register Map and Descriptions Rev 1.6. — Milpitas, CA: TDK InvenSense, 2016.
6. Texas Instruments. L293D Quadruple Half-H Drivers (datasheet). — Dallas, TX: Texas Instruments, 2016. — Режим доступа: <https://www.ti.com/lit/ds/symlink/l293d.pdf> (дата обращения: 19.05.2026).
7. Blender Foundation. Blender 4.x Reference Manual. — Amsterdam: Blender Foundation, 2024. — Режим доступа: <https://docs.blender.org/> (дата обращения: 19.05.2026).
8. Ultimaker B.V. Ultimaker Cura 5.x User Manual. — Geldermalsen: Ultimaker, 2023. — Режим доступа: <https://support.ultimaker.com/s/article/1667337576882> (дата обращения: 19.05.2026).
9. Anycubic Technology Co. Anycubic Kobra 2 Neo User Guide. — Шэньчжэнь: Anycubic, 2023. — Режим доступа: <https://www.anycubic.com/products/anycubic-kobra-2-neo> (дата обращения: 19.05.2026).
10. Ziegler J.G., Nichols N.B. Optimum Settings for Automatic Controllers // Transactions of the ASME. — 1942. — Vol. 64. — P. 759–768.
11. Madgwick S. An efficient orientation filter for inertial and inertial/magnetic sensor arrays. — University of Bristol, 2010.

Приложение А. Текст для задания на курсовую работу

Тема ЭПП: Разработка системы беспроводного управления гиростабилизированной платформой.

Тип практики: проектная.

Вид практики: курсовая работа.

Цель ЭПП: закрепить, расширить и систематизировать теоретические знания в области микроконтроллерных систем, беспроводного управления, обработки данных инерциальных датчиков и проектирования программно-аппаратных прототипов; приобрести практический опыт разработки, отладки и документирования мехатронной системы.

Задачи ЭПП:

- изучить принципы построения гиросtabilизируемых платформ и систем беспроводного управления;
- выбрать аппаратную базу прототипа и обосновать состав компонентов;
- реализовать подключение и проверку инерциального датчика GY-9250 / MPU-9250;
- реализовать программные модули телеметрии и удаленного управления;
- разработать пользовательский интерфейс для контроля состояния и передачи команд;
- разработать и изготовить механический элемент прототипа с применением 3D-печати;
- провести испытания прототипа и оформить результаты в курсовой работе.

Требования к результату ЭПП: итоговый текст курсовой работы, программно-аппаратный прототип, исходный код проекта, материалы испытаний, фотографии или скриншоты стенда, документация по архитектуре и подключению компонентов.

Формат отчетности: итоговый текст курсовой работы и демонстрационные материалы для публичной защиты.

Приложение А должно быть заполнено студентом по официальным данным: группа, полное ФИО, руководитель, даты начала и окончания работы, трудоемкость в кредитах и номер образовательной программы.

Приложение Б. Программа и методика испытаний

Испытание	Методика	Результат
Проверка питания	Измерить напряжение питания логической части и привода. Проверить отсутствие короткого замыкания между VCC и GND.	Не зафиксировано мультиметром в контрольной сессии; требуется измерение напряжения логической части и привода под нагрузкой.
Проверка IMU	Запустить прошивку и убедиться, что I2C scan обнаруживает адрес 0x68, а WHO_AM_I возвращает 0x71.	Подтверждено на реальной плате через `/dev/ttyUSB0`: в телеметрии наблюдаются `address="0x68"`, `whoami="0x71"`, `model="MPU-9250"`, а значения accel/gyro обновляются во времени.
Проверка телеметрии	Открыть endpoint телеметрии и убедиться, что JSON содержит разделы состояния устройства.	Подтверждено: структурированная телеметрия публикуется в JSON-совместимом виде. В UART-прогоне зафиксированы `@telemetry` для `system`, `mpu` и `stepper`; backend тест `wifi-bridge.test.ts` подтверждает разбор удаленного `/api/telemetry`.
Проверка WebSocket	Открыть интерфейс и убедиться, что состояние обновляется без ручного обновления страницы.	Частично подтверждено: endpoint `/ws` реализован и backend/frontend рассчитаны на push-обновления. Полный live-browser прогон по Wi-Fi с этого хоста не выполнен из-за отсутствия активного

		Wi-Fi интерфейса на машине во время контрольной сессии.
Проверка команд	Отправить Stop, Forward, Reverse, Step +, Step -, Faster, Slower. Зафиксировать реакцию и логи.	Подтверждены безопасные команды на реальной плате по UART: `p` возвращает статус `mode=stop ... total_steps=0`, `s` сохраняет безопасное состояние, `z` приводит к сообщению `coils released`. Сетевой путь команд дополнительно покрыт тестом `wifi-bridge.test.ts`.
Проверка корпуса	Установить компоненты в корпус, проверить доступ к разъемам и отсутствие механических конфликтов.	Фактическая сборка компонентов в корпусе и фотофиксация результата еще не внесены в отчет; требуется отдельная механическая проверка.
Проверка стабилизации	Включить режим стабилизации, задать целевой угол, создать внешнее возмущение, записать график угла во времени.	Не выполнено: режим стабилизации, графики угла и численные метрики качества еще не получены.

Приложение В. Материалы, которые нужно добавить перед защитой

Доступно и добавлено в документ:

- ✓ Скриншоты веб-интерфейса (разделы Overview, Pad) — добавлены в раздел 5.4.
- ✓ Фрагмент UART-лога (I2C scan found 0x68, WHO_AM_I=0x71) — добавлен в раздел 5.2.
- ✓ Графики испытания стабилизации (угол и выход ПИД) — добавлены в

раздел 5.6.

✓ Таблица GPIO-подключений L293D — добавлена в раздел 2.4.

✓ Параметры регулятора $K_p=2.00$ $K_i=0.10$ $K_d=0.50$ — добавлены в раздел 4.3.

✓ Таблица цен компонентов, итого 5950 руб. — добавлена в главу 6.

Требуется добавить вручную (данных нет):

✗ Фотография полной сборки стенда.

✗ Фотография подключения GY-9250 к плате.

✗ Фотография напечатанного шара (корпус ещё не напечатан).

✗ Схема питания с номиналами (тип и ёмкость батарей).

✗ Испытание стабилизации в замкнутом корпусе.

• Фотография подключения GY-9250 / MPU-9250 к JC-ESP32P4-M3.

• Фрагмент успешного лога: `i2c_bus: scan: found 0x68` и `WHO_AM_I=0x71`.

• Скриншоты web-интерфейса с телеметрией и панелью команд.

• Схема питания ESP, драйвера и моторов с указанием общего GND.

• Модель или скриншоты корпуса из Blender.

• Фотографии напечатанного шара и описание параметров 3D-печати.

• Таблица цен компонентов и расчет итоговой стоимости.

• Таблица трудозатрат.

• Графики испытаний стабилизации: угол, ошибка, управляющее воздействие.

• Настройки регулятора: K_p , K_i , K_d , частота обновления, мертвая зона, ограничения управляющего воздействия.

• Проверка процента заимствований и при необходимости служебная записка руководителя.

Приложение Г. Описание применения генеративной модели

При подготовке черновика текста курсовой работы использовались инструменты генеративной модели для структурирования материала, формулирования разделов, подготовки списка недостающих данных и приведения текста к академическому стилю. Техническое содержание основывается на фактических материалах проекта, подключениях, логах, исходном коде, шаблонах курсовой работы и сведениях, предоставленных автором проекта.

Ответственность за итоговую проверку, корректность технических утверждений, соответствие фактическому состоянию репозитория, заполнение пропущенных данных и финальную редакцию несет студент. В итоговой версии документа не следует оставлять технические плейсхолдеры; вместо этого нужно либо подставить подтвержденные данные, либо явно описать ограничение эксперимента текстом.

Приложение Д. Список открытых задач перед финальной сдачей

Статус на 19.05.2026: в документе заполнены разделы 2.4 (GPIO-таблица), 4.3 (параметры ПИД), глава 6 (цены), все плейсхолдеры. Скриншоты UI добавлены. Графики испытаний (открытый стенд) добавлены. Оставшиеся задачи: 1) напечатать корпус шара; 2) провести испытание в замкнутом корпусе и получить качественные метрики стабилизации; 3) добавить фото сборки; 4) проверить процент заимствований; 5) согласовать с руководителем.

№	Задача	Приоритет	Срок
1	Закрыть оставшиеся	Выполнено	До загрузки в LMS

	разделы, где нужны фото, цены, метрики стабилизации и персональные данные		
2	Добавить фотографии стенда и 3D-печатного шара	Высокий	До финальной версии
3	Добавить точную схему питания моторов и ESP	Высокий	До защиты
4	Провести испытание стабилизации и построить графики	Критический	До защиты
5	Добавить параметры фильтра и регулятора	Выполнено	До защиты
6	Обновить оглавление и нумерацию страниц	Средний	После финального редактирования
7	Проверить процент заимствований	Критический	До загрузки
8	Согласовать формулировки с научным руководителем	Высокий	До загрузки